CS2 — Data Structures

Programming Assignment

SLO 1 Assessment

1. Learning Objective

SLO1 - Students will **design, code, compile, test, and document** a programming solution to a problem involving the basic data structures: lists, stacks, queues, trees, graphs, hash tables, and related abstract data types.

This assignment targets the **linked list** — one of the most fundamental sequential data structures in computer science. You will design a `Person` class with validated setters, then build an interactive database that stores Person objects in an STL `list<Person>` and demonstrates the operations characteristic of a doubly-linked list.

2. Part 1 — Design the Person Class

Implement the `Person` class in a file named `Person.h`. The class must include **setters with basic input validation** — setters that reject invalid values by throwing a `runtime_error` exception rather than silently storing bad data.

2.1 Required Members

Member	Description
name (private, string)	The person's full name.
age (private, int)	The person's age.
setName(string)	Validates: rejects empty string. Throws runtime_error if invalid.
setAge(int)	Validates: rejects age < 0 or age > 120. Throws runtime_error if invalid.
getName() const	Accessor — returns name.
getAge() const	Accessor — returns age.
Person(string, int)	Constructor — calls setName() and setAge(). Default args: "n.a", 0.
toString() const	Returns formatted string. Declared virtual for future inheritance.
operator<<	Friend function — delegates to toString(). Takes const Person&.

2.2 Starter Code

```
// Person.h
#pragma once
#include <iostream>
#include <sstream>
#include <stdexcept>
#include <string>
using namespace std;

class Person {
private:
    string name;
    int    age;

public:
    //-- Setters with validation ---------------------
    void setName(const string& nameValue) {
        // TODO: throw runtime_error if nameValue is empty
        name = nameValue;
    }

    void setAge(int ageValue) {
        // TODO: throw runtime_error if ageValue < 0 or ageValue > 120
        age = ageValue;
    }

    //-- Getters -----------------------------------
    string getName() const { return name; }
    int    getAge()  const { return age;  }

    //-- Constructor -------------------------------
    Person(string n = "n.a", int a = 0) {
        setName(n);
        setAge(a);
    }

    //-- toString ----------------------------------
    virtual string toString() const;

    //-- operator<< --------------------------------
    friend ostream& operator<<(ostream& sout, const Person& p) {
        sout << p.toString();
        return sout;
    }
};

string Person::toString() const {
    ostringstream oss;
    oss << this << " Person [Name: " << name
        << ", Age: " << age << "]";
    return oss.str();
}
```

Validation rules to implement in the setters:

- `setName()` — throw `runtime_error("Name cannot be empty.")` if the string is empty.
- `setAge()` — throw `runtime_error("Age out of range: " + to_string(ageValue))` if age is below 0 or above 120.

The constructor calls both setters, so any invalid argument at construction time automatically throws and prevents a bad object from being created.

3. Part 2 — Build the Interactive Database

In `main.cpp` implement an input loop that reads Person data from the keyboard, creates Person objects, and stores them in a `list<Person>` named `db`.

3.1 Input Loop Requirements

The loop must:

- Prompt for a name using `getline(cin, name)`.
- Prompt for an age using `cin >> age` followed by `cin.ignore(numeric_limits<streamsize>::max(), '\n')` to flush the buffer.
- Wrap the Person construction in a `try-catch` block — if a setter throws, print the error and ask the user to try again **without** adding a bad record to the list.
- Add the valid Person to the list using `db.push_back(...)` or `db.emplace_back(...)`.
- After each successful entry ask: `"Add another person? [y/n]: "`.
- After the loop, print the complete list using a range-for loop and report `db.size()`.

3.2 Starter Code

```
void buildDatabase(list<Person>& db) {
    cout << "=== Person Database  list<Person> ===\n";
    char again = 'y';
    string name;
    int    age;

    while (tolower(again) == 'y') {
        cout << "Enter name: ";
        getline(cin, name);

        cout << "Enter age:  ";
        cin  >> age;
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        try {
            db.push_back(Person(name, age));
            cout << "  Added: " << db.back() << "\n";
        }
        catch (const runtime_error& e) {
            cout << "  REJECTED: " << e.what() << "\n";
        }

        cout << "Add another? [y/n]: ";
        cin  >> again;
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}
```

4. Part 3 — Demonstrate List-Specific Operations

After the interactive loop, demonstrate the three operations that are characteristic of a doubly-linked list. Print the list after each operation to confirm the change.

Operation	What to do
push_front()	Add one hardcoded Person to the FRONT of the list.
pop_front()	Save front() to a variable, print it, then call pop_front() to remove it.
pop_back()	Save back() to a variable, print it, then call pop_back() to remove it.

```
void demonstrateListOps(list<Person>& db) {
    cout << "\n--- Part 3: List Operations ---\n";

    // push_front() -- add to the front
    db.push_front(Person("Montgomery Burns", 104));
    cout << "After push_front(Burns)  size=" << db.size() << ":\n";
    for (const auto& p : db) cout << "  " << p << "\n";

    // pop_front() -- remove from the front
    // ALWAYS save the value BEFORE calling pop -- it returns void!
    Person removed_front = db.front();
    db.pop_front();
    cout << "\npop_front() removed: " << removed_front << "\n";
    cout << "After pop_front()  size=" << db.size() << ":\n";
    for (const auto& p : db) cout << "  " << p << "\n";

    // pop_back() -- remove from the back
    Person removed_back = db.back();
    db.pop_back();
    cout << "\npop_back() removed: " << removed_back << "\n";
    cout << "After pop_back()  size=" << db.size() << ":\n";
    for (const auto& p : db) cout << "  " << p << "\n";
}
```

5. Sample Interaction

```
=== Person Database  list<Person> ===
Enter name: Homer Simpson
Enter age:  45
  Added: 0x... Person [Name: Homer Simpson, Age: 45]
Add another? [y/n]: y

Enter name: Marge Simpson
Enter age:  42
  Added: 0x... Person [Name: Marge Simpson, Age: 42]
Add another? [y/n]: y

Enter name:
Enter age:  25
  REJECTED: Name cannot be empty.
Add another? [y/n]: y

Enter name: Lisa Simpson
Enter age:  8
  Added: 0x... Person [Name: Lisa Simpson, Age: 8]
Add another? [y/n]: n

--- Database (3 records) ---
0x... Person [Name: Homer Simpson, Age: 45]
0x... Person [Name: Marge Simpson, Age: 42]
0x... Person [Name: Lisa Simpson,  Age: 8 ]

--- Part 3: List Operations ---
After push_front(Burns)  size=4:
  0x... Person [Name: Montgomery Burns, Age: 104]
  0x... Person [Name: Homer Simpson,    Age: 45 ]
  0x... Person [Name: Marge Simpson,    Age: 42 ]
  0x... Person [Name: Lisa Simpson,     Age: 8  ]

pop_front() removed: 0x... Person [Name: Montgomery Burns, Age: 104]
After pop_front()  size=3:
  ...

pop_back() removed: 0x... Person [Name: Lisa Simpson, Age: 8]
After pop_back()  size=2:
  0x... Person [Name: Homer Simpson, Age: 45]
  0x... Person [Name: Marge Simpson, Age: 42]
```

6. Deliverables

File	Contents
Person.h	Complete Person class with validated setName(), setAge(), toString(), and operator<<.
main.cpp	buildDatabase(), demonstrateListOps(), and main() calling both.

7. Self-Assessment: Quality Control

Before submitting your work review each item below. A check on every line means your assignment is complete.

☐	Item
☐	Person.h compiles with no errors or warnings.
☐	setName() throws runtime_error when name is empty.
☐	setAge() throws runtime_error when age is below 0 or above 120.
☐	Constructor calls setName() and setAge() so bad arguments throw at construction time.
☐	toString() is declared virtual and operator<< takes const Person&.
☐	Input loop uses getline() for name and cin >> for age.
☐	cin.ignore() called after every cin >> to prevent buffer contamination.
☐	try-catch wraps each Person construction — bad records are rejected, not added to the list.
☐	Objects stored in a list<Person> using push_back() or emplace_back().
☐	List printed with a range-for loop using operator<<.
☐	push_front(), pop_front(), and pop_back() all demonstrated.
☐	Value saved from front() or back() BEFORE calling pop_front() or pop_back().
☐	List printed after each modification to confirm the change.
☐	db.size() displayed after each operation.

Key concept

A linked list excels at insertion and deletion at its ends. Unlike a vector, adding or removing from the front is O(1) — no elements are shifted. This is the structural advantage that makes `list` the right container whenever your program frequently modifies both ends of a sequence.

Programming Style Note: A member function defined inside a class definition is implicitly *inline*. This does not mean that the compiler must replace every function call with the function's code. Whether a function call is actually *inlined* is an optimization decision made by the compiler.